



Digital Design

Chapter 2: Combinational Logic Design

Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

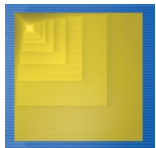
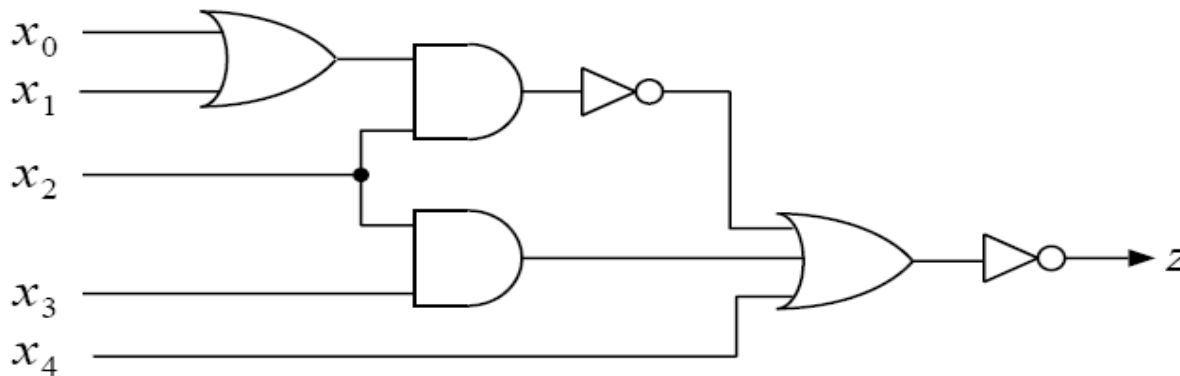
Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

Completeness of Logic Gates

- Any Boolean function can be implemented using AND, OR and NOT because a Boolean function can be represented by a **sum of products (use truth table)**, which consists only of AND, OR and NOT.

Set {AND,OR,NOT}

$$z = (((x_0 + x_1)x_2)' + x_2x_3 + x_4)'$$

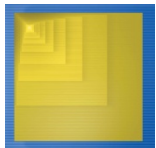
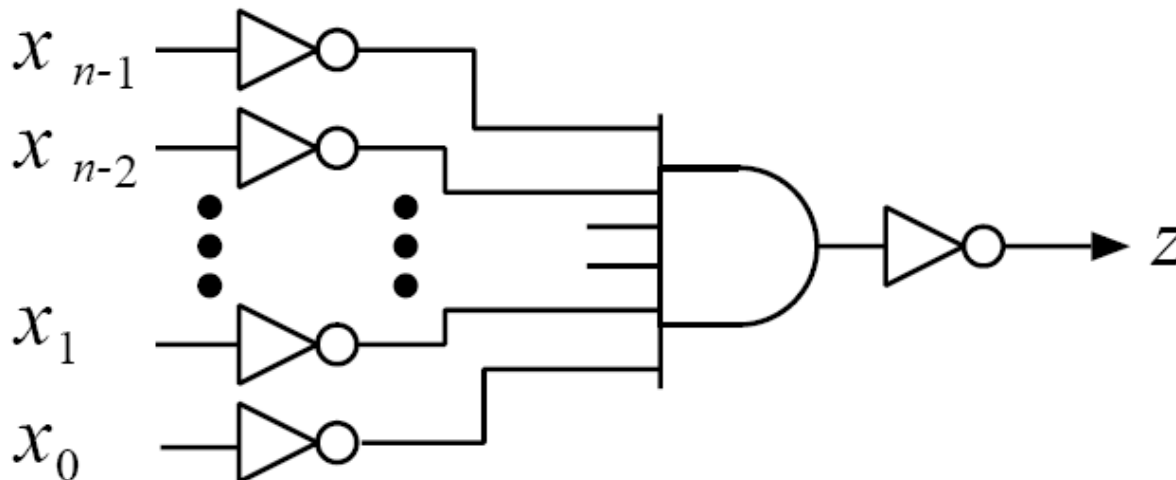
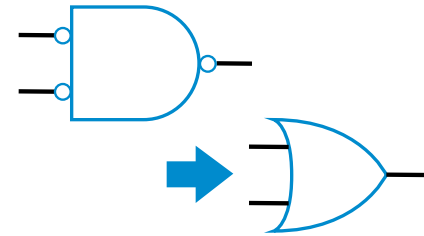


Completeness of Logic Gates

- Any Boolean function can be implemented using AND and NOT. Put NOT gates at the inputs and outputs of an AND gate to obtain OR gate.
- Similarly, for OR and NOT.

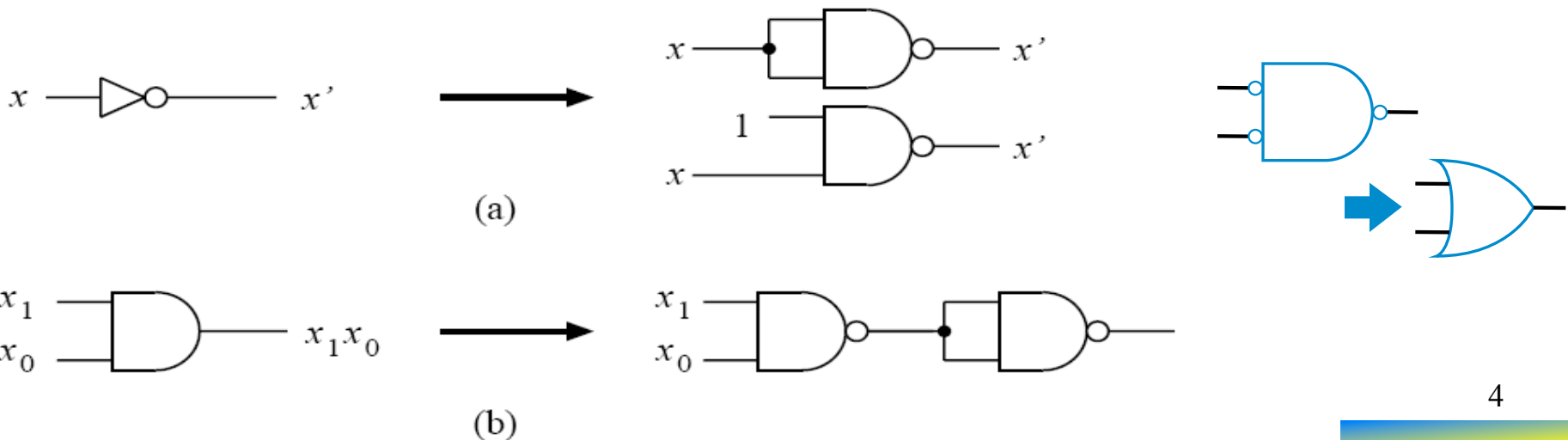
Sets {AND, NOT} and {OR, NOT}

$$x_{n-1} + x_{n-2} + \dots + x_i + \dots + x_0 = (x'_{n-1} x'_{n-2} \dots x'_i \dots x'_0)'$$

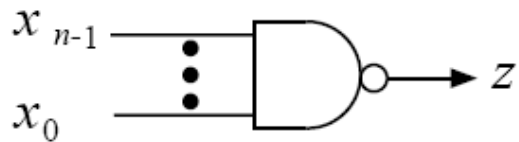


Completeness of NAND, NOR

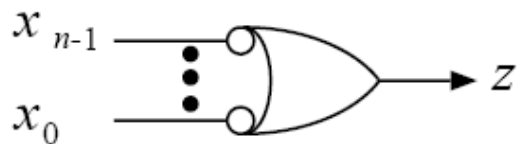
- Any Boolean function can be implemented *using just NAND gates*. Why?
 - Can implement NOT, AND, and OR with NAND
 - NOT: 1-input NAND (or 2-input NAND with inputs tied together)
 - AND: NAND followed by 1-input NAND
 - OR: NAND preceded by NOTs
- Likewise for NOR. Both NAND and NOR gates are known as *universal gates*.



Mixed Logic Notation

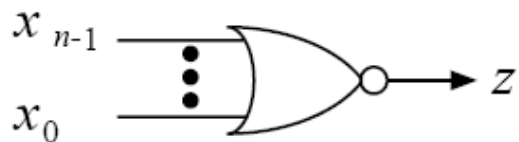


$$Z = (x_{n-1} \dots x_1 x_0)'$$

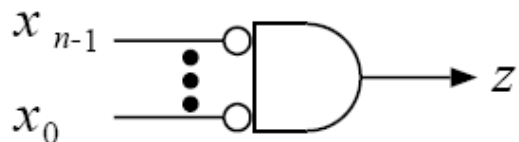


$$Z = x'_{n-1} + \dots + x'_1 + x'_0$$

(a)

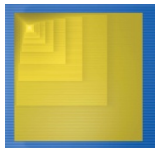


$$Z = (x_{n-1} + \dots + x_1 + x_0)'$$



$$Z = x'_{n-1} \dots x'_1 x'_0$$

(b)



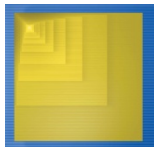
Number of Possible Boolean Functions

- How many possible functions of 2 variables?
 - 2^2 rows in truth table, 2 choices for each
 - $2^{(2^2)} = 2^4 = 16$ possible functions
- N variables
 - 2^N rows
 - $2^{(2^N)}$ possible functions

a	b	F
0	0	0 or 1 2 choices
0	1	0 or 1 2 choices
1	0	0 or 1 2 choices
1	1	0 or 1 2 choices

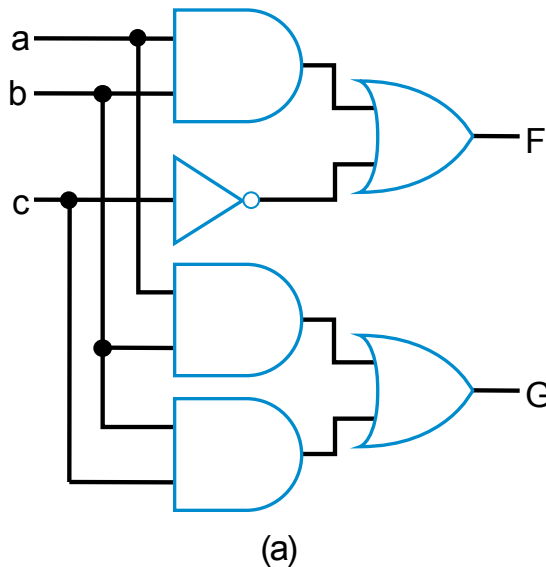
$2^4 = 16$
possible functions

a	b	f0	f1	f2	f3	f4	f5	f6	f7	f8	f9	f10	f11	f12	f13	f14	f15
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
		0	a AND b		a		b	a XOR b	a OR b	a NOR b	a XNOR b	b'		a'		a NAND b	1

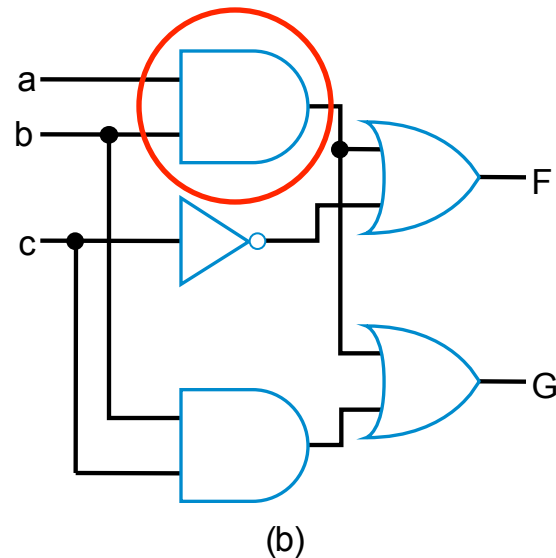


Multiple-Output Circuits

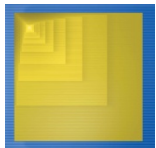
- Many circuits have more than one output
- Can give each a separate circuit, or can share gates
- Ex: $F = \underline{ab} + c'$, $G = \underline{ab} + bc$



Option 1: Separate circuits



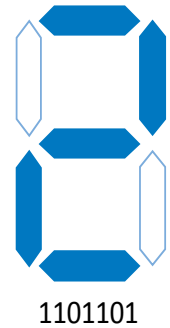
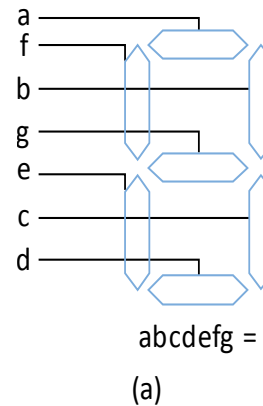
Option 2: Shared gates



Multiple-Output Example: BCD to 7-Segment Converter

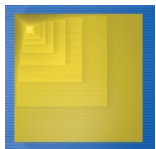
TABLE 2-4 4-bit binary number to seven-segment display truth table

w	x	y	z	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0



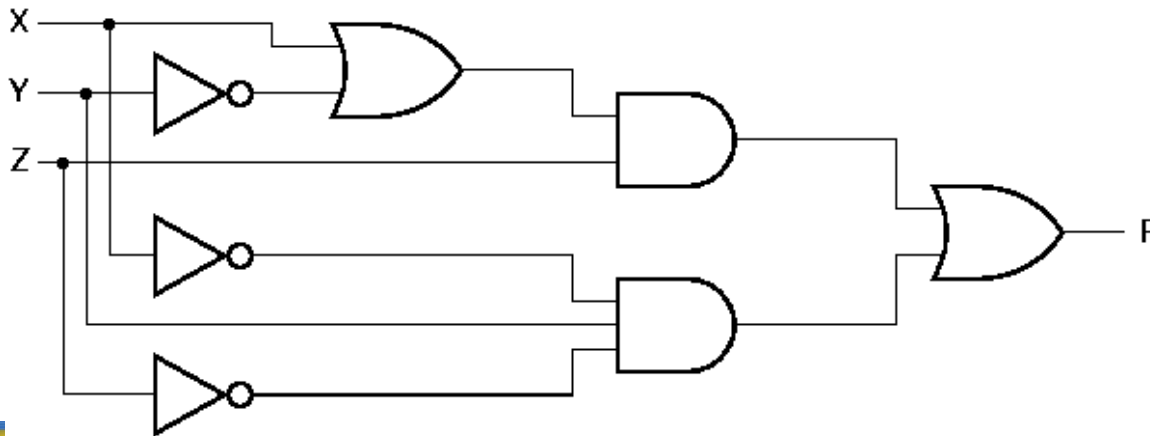
$$a = w'x'y'z' + w'x'yz' + w'x'yz + w'xy'z + w'xyz' + w'xyz + wx'y'z' + wx'y'z$$

$$b = w'x'y'z' + w'x'y'z + w'x'yz' + w'x'yz + w'xy'z' + w'xyz + wx'y'z' + wx'y'z$$



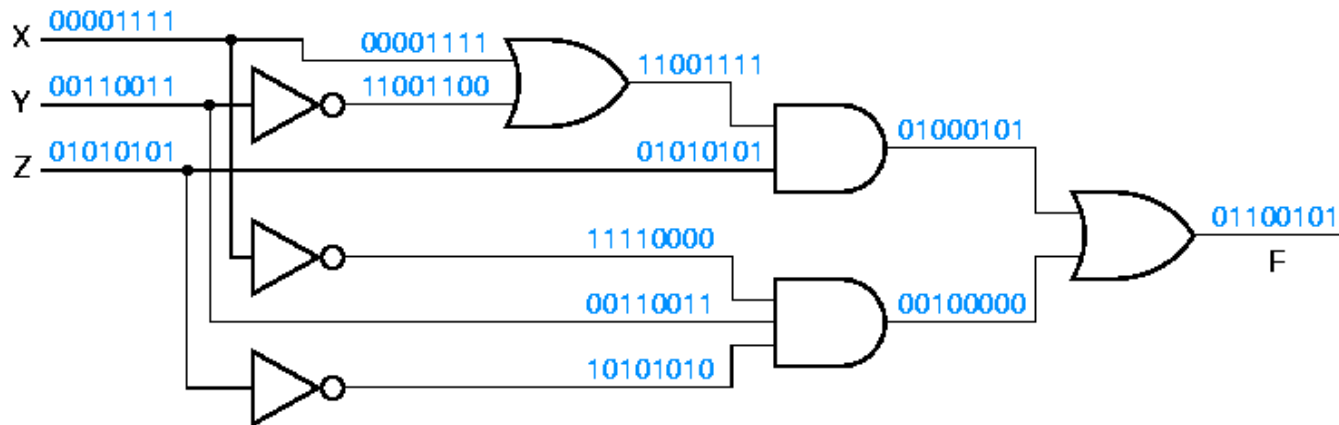
Combinational Analysis

- We can analyze a circuit by obtaining a formal description of its logic function. After having a formal description of the logic function, we can:
 - Determine the behaviour of the circuit for various input combinations.
 - Manipulate the algebraic description to suggest different circuit structures.
 - Transform the algebraic description into a standard form.
 - Use the algebraic description of the circuit to analyse a larger system that includes it as a subsystem.
- **Example:** Analyze the following combinational circuit



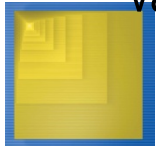
Combinational Analysis

- We can obtain the truth table by going through all input combinations.



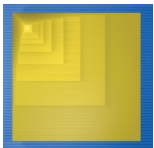
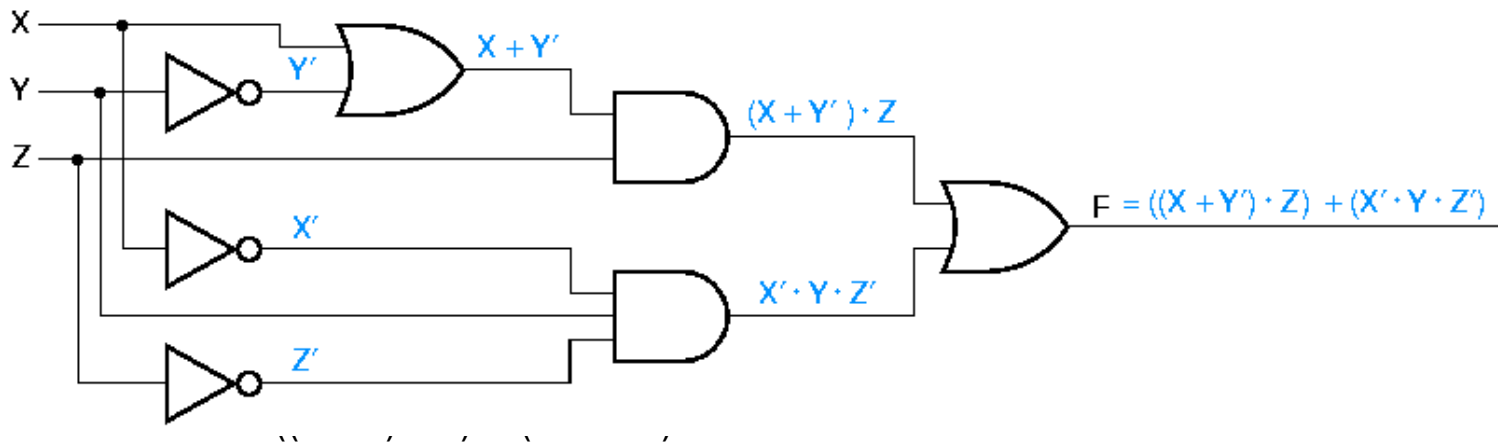
Row	X	Y	Z	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

- From the truth table we can directly write a logic expression.
- This technique is time-consuming and only works for small number of input variables.



Combinational Analysis

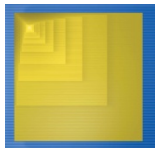
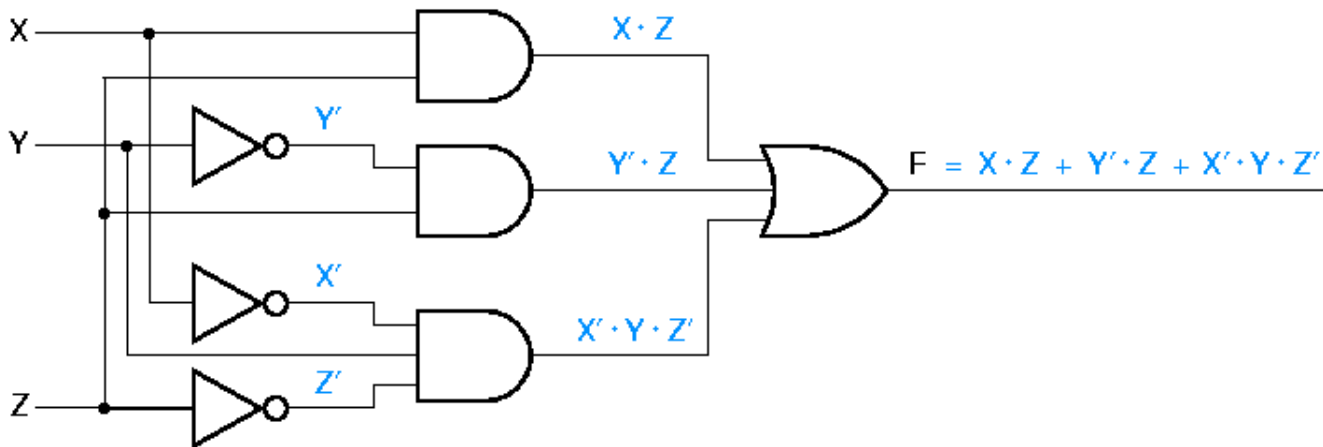
- Generate expressions starting from inputs to outputs



Combinational Analysis

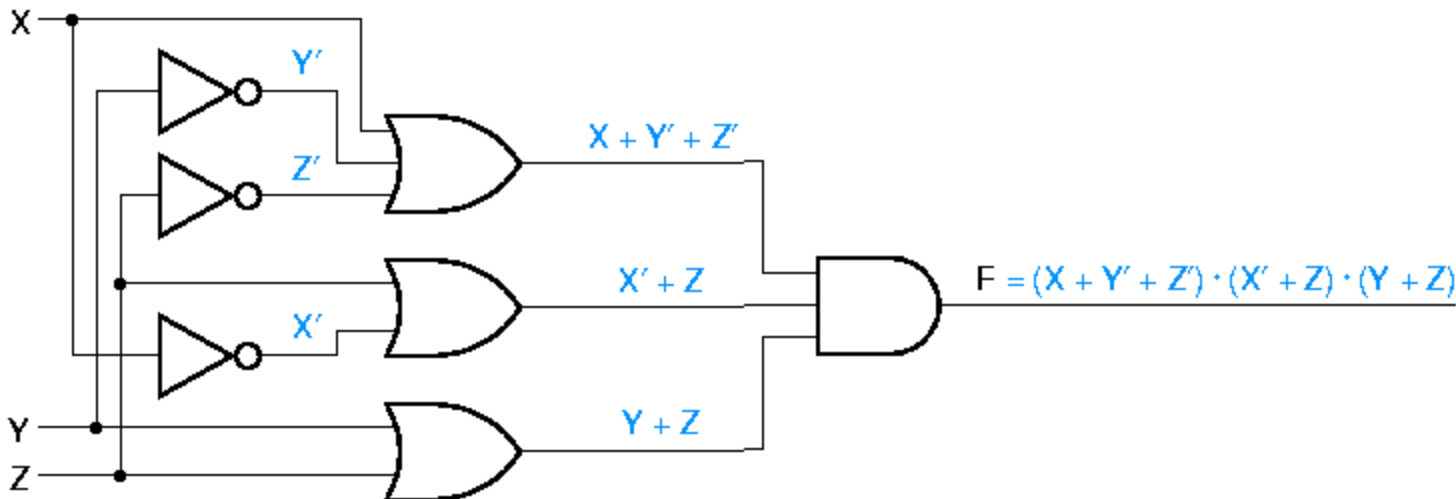
- After some algebraic transformation, F is in SOP form.
- $F = ((X+Y') \cdot Z) + (X' \cdot Y \cdot Z')$
 $= (X \cdot Z) + (Y' \cdot Z) + (X' \cdot Y \cdot Z')$ (distributive law)

Sum of products - Two level AND-OR circuit



Combinational Analysis

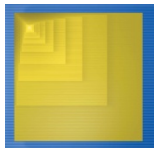
- We can obtain a product of sums term from the original expression.
 - $$\begin{aligned} F &= ((X+Y') \cdot Z) + (X' \cdot Y \cdot Z') \\ &= ((X + Y') + (X' \cdot Y \cdot Z')). (Z + (X' \cdot Y \cdot Z')) \text{ (distributive law)} \\ &= (X+Y'+X') \cdot (X+Y'+Y) \cdot (X+Y'+Z') \cdot (Z+X') \cdot (Z+Y) \cdot (Z+Z') \text{ (distributive law)} \\ &= 1 \cdot 1 \cdot (X+Y'+Z') \cdot (Z+X') \cdot (Z+Y) \cdot 1 \text{ (complement)} \\ &= (X+Y'+Z') \cdot (X'+Z) \cdot (Y+Z) \end{aligned}$$
 - Two level OR-AND circuit



Combinational Logic Design (Synthesis) Process

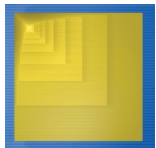
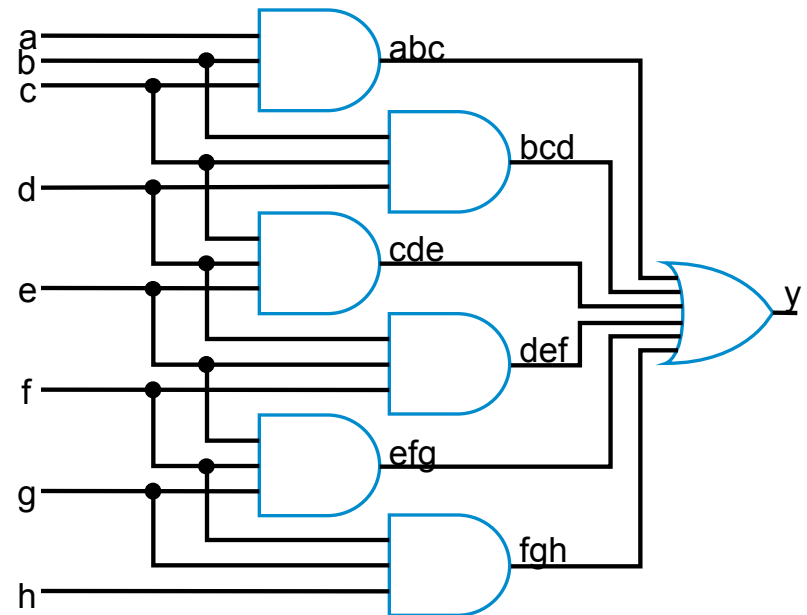
Most of the time, synthesis is done automatically for the designer. Other times, for optimization purposes (performance, power, size), manual synthesis may be done.

Step		Description
Step 1	Capture the function	Create a truth table or equations, whichever is most natural for the given problem , to describe the desired behavior of the combinational logic.
Step 2	Convert to equations	This step is only necessary if you captured the function using a truth table instead of equations. Create an equation for each output by ORing all the minterms for that output. Simplify the equations if desired.
Step 3	Implement as a gate-based circuit	For each output, create a circuit corresponding to the output's equation. (Sharing gates among multiple outputs is OK optionally.)



Example: Three 1s Detector

- **Problem:** Detect three consecutive 1s in 8-bit input: abcdefgh
 - 00011101 $\rightarrow$ 1 10101011 $\rightarrow$ 0
 - 11110000 $\rightarrow$ 1
- **Step 1: Capture** the function
 - Truth table or equation?
 - Truth table too big: $2^8=256$ rows
 - Equation: create terms for each possible case of three consecutive 1s
 - $y = abc + bcd + cde + def + efg + fgh$
- **Step 2: Convert** to equation -- already done
- **Step 3: Implement** as a gate-based circuit



Example: Number of 1s Count

- Problem:** Output in binary on two outputs **yz** the number of 1s on three inputs

- 010 → 01 101 → 10 000 → 00

- **Step 1: Capture** the function

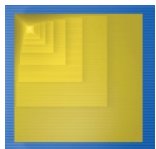
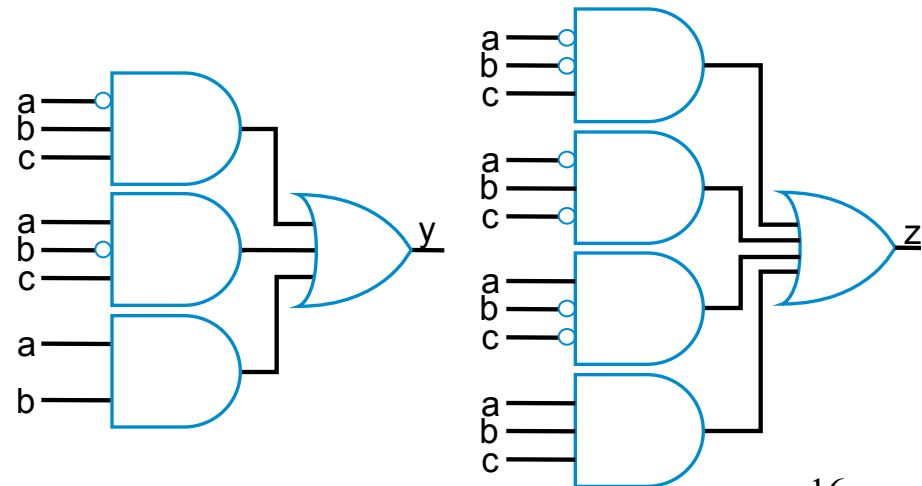
- Truth table or equation?
 - Truth table is straightforward

- **Step 2: Convert** to equation

- $y = a'bc + ab'c + abc' + abc$
 - $z = a'b'c + a'bc' + ab'c' + abc$

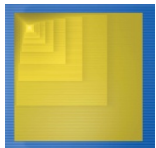
- **Step 3: Implement** as a gate-based circuit

Inputs			# of 1s	Outputs	
a	b	c		y	z
0	0	0	(0)	0	0
0	0	1	(1)	0	1
0	1	0	(1)	0	1
0	1	1	(2)	1	0
1	0	0	(1)	0	1
1	0	1	(2)	1	0
1	1	0	(2)	1	0
1	1	1	(3)	1	1



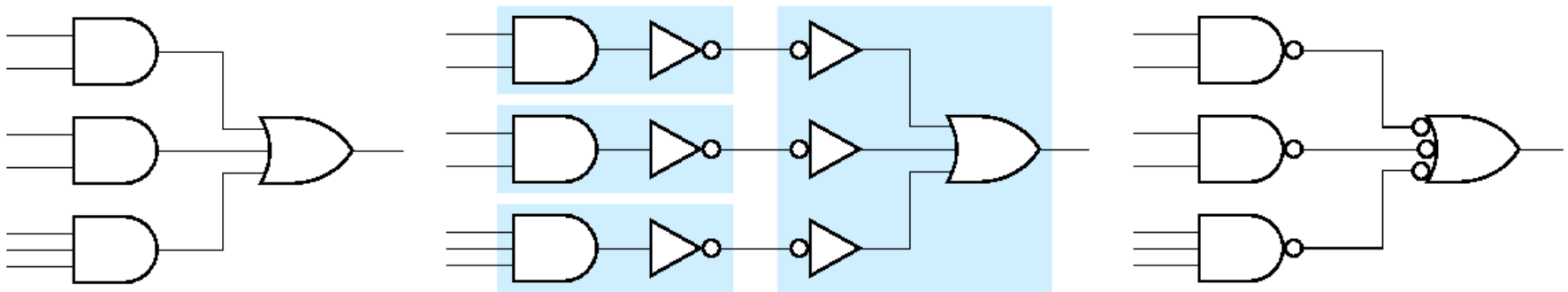
Combinational Circuit Manipulations

- We have described methods that use AND, OR, and NOT gates.
- In some situations, we might like to use NAND or NOR gates (they are faster than ANDs and ORs in most technologies).
- However, most people don't develop logic propositions in terms of NAND and NOR connectives.
 - Nobody says: "I don't like a girl, if she is not smart or not pretty and also if she is not rich or not friendly". $[G' = (S' + P') \cdot (R' + F')]$
 - It is more common to say: "I like a girl, if she is smart and pretty or if she is rich and friendly". $[G = (S \cdot P) + (R \cdot F)]$
- Any logic expression can be transformed into an equivalent sum-of-products (SOP) expression, simply by multiplying it out and implemented with AND and OR gates.

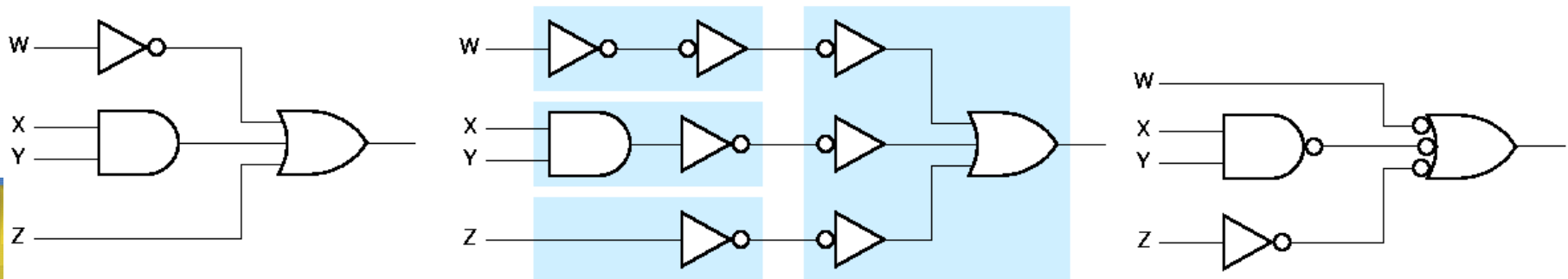


Combinational Circuit Manipulations

- A 2-level AND-OR circuit may be converted to a 2-level NAND-NAND circuit simply by substituting gates.

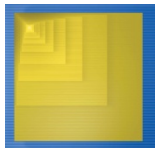
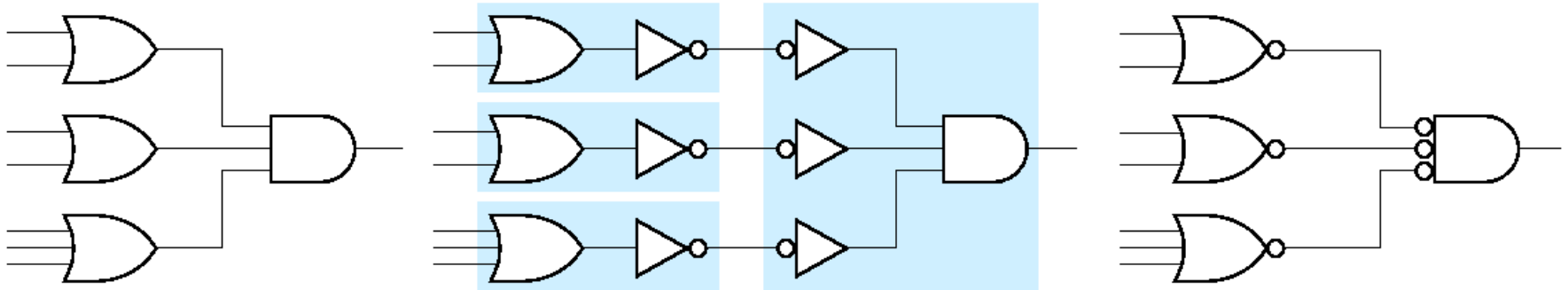


If any product terms in the sum-of-products expression contains just a single literal, inverters may appear or disappear.



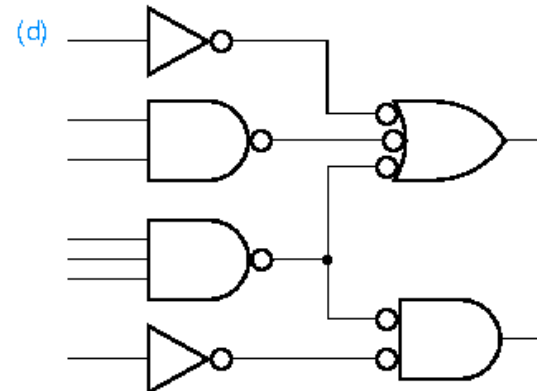
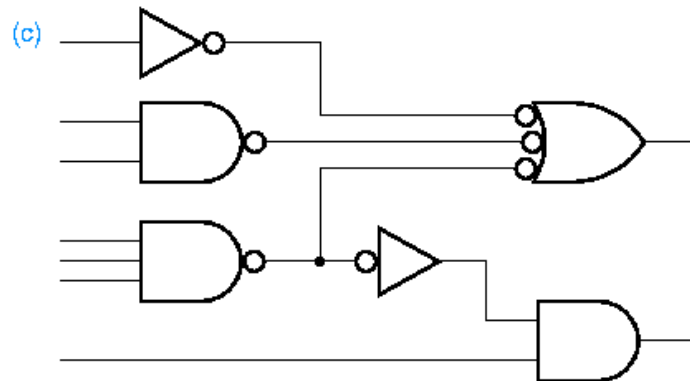
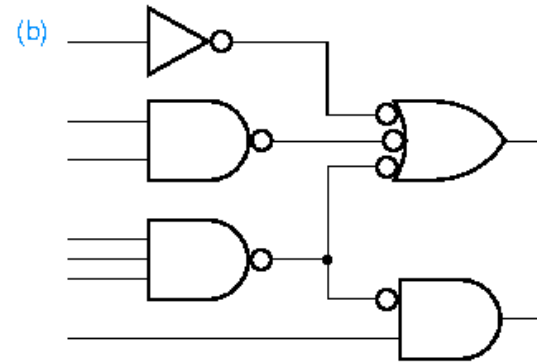
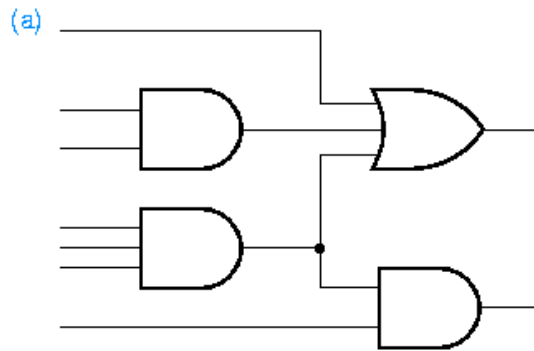
Combinational Circuit Manipulations

- Any sum-of-products (SOP) expression can be realised in either two ways (AND-OR circuit or NAND-NAND circuit).
- The principle of duality can be applied to this rule:
Any product-of-sums (POS) expression can be realised in either two ways (OR-AND circuit or NOR-NOR circuit).



Combinational Circuit Manipulations

- These manipulations can be applied to arbitrary logic circuits.

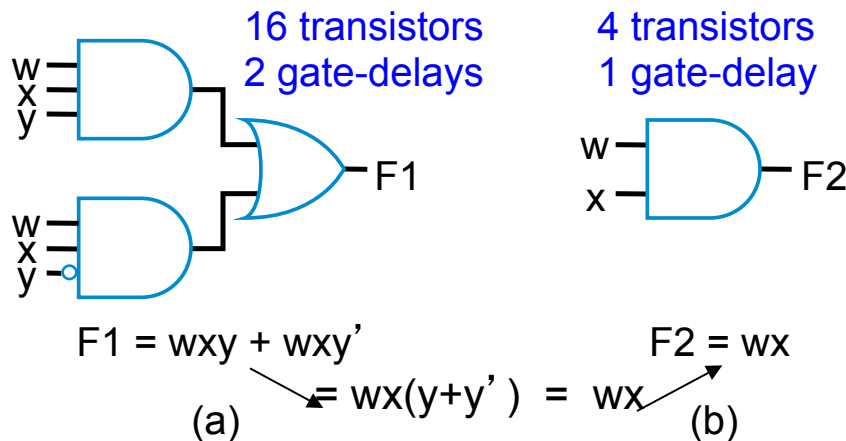


The (d) solution is better than the (c) solution.

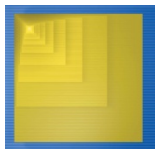
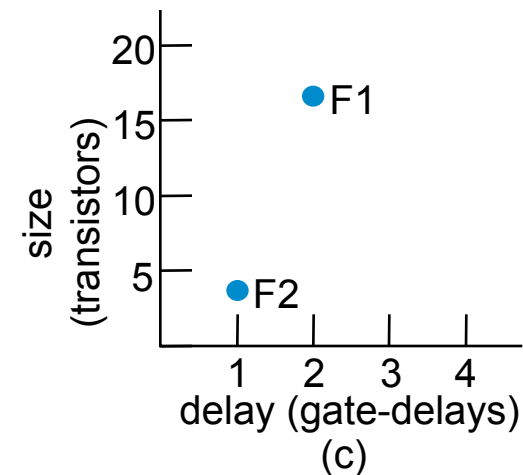


Minimization

- We now know how to build digital circuits
 - How can we build **better** circuits?
- Let's consider two important design criteria
 - **Delay** – the time from inputs changing to new correct stable output
 - **Size** – the number of transistors
 - For quick estimation, assume
 - Every gate has delay of “1 gate-delay”
 - Every gate *input* requires 2 transistors
 - Ignore inverters



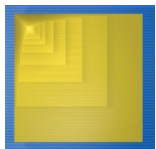
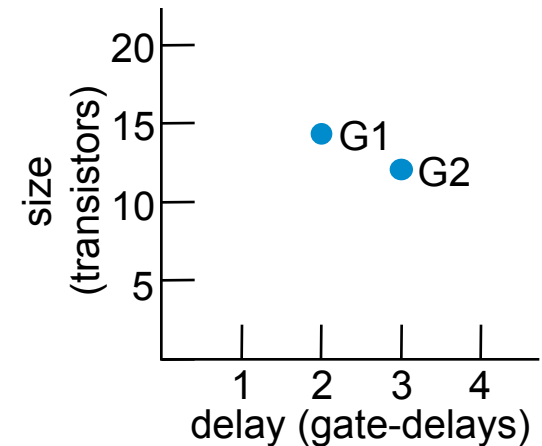
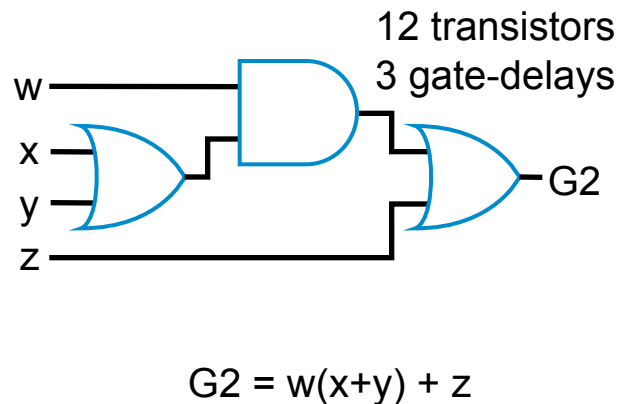
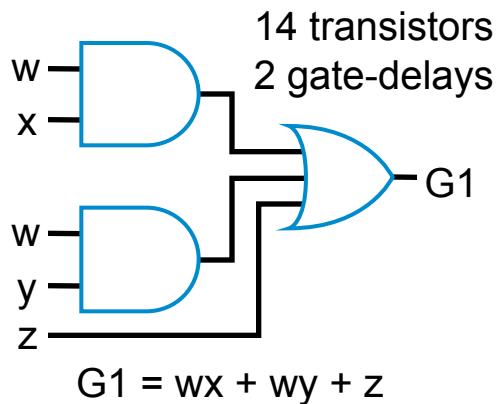
Transforming F1 to F2 represents an **optimization**: Better in all criteria of interest



Minimization

- Tradeoff
 - Improves some, but worsens other, criteria of interest

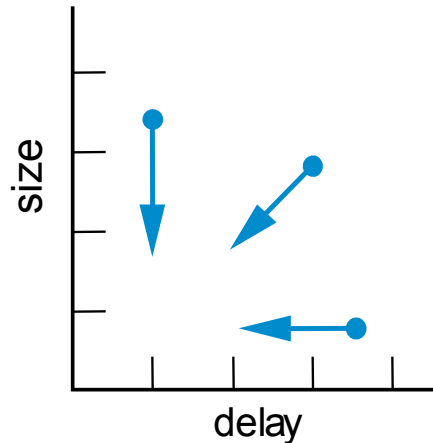
Transforming G1 to G2 represents a **tradeoff**. Some criteria better, others worse.



Minimization

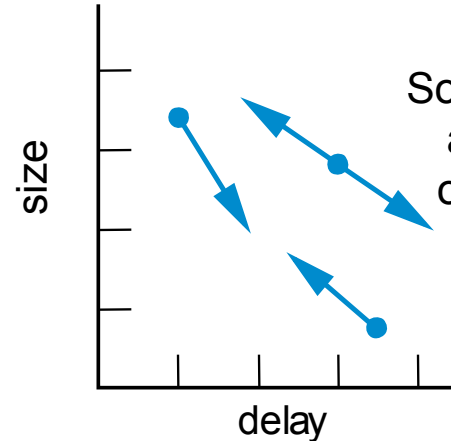
Optimizations

All criteria of interest are improved (or at least kept the same)

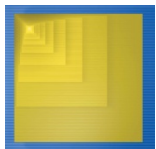


Tradeoffs

Some criteria of interest are improved, while others are worsened

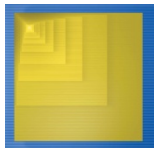


- We obviously prefer optimizations, but often must accept tradeoffs
 - You can't build a car that is the most comfortable, and has the best fuel efficiency, and is the fastest – you have to give up something to gain other things.



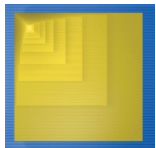
Combinational Circuit Minimization

- It is uneconomical to realise a logic function directly from the first expression that comes up.
- Canonical (sum and product) expressions are especially expensive.
- Minimization techniques reduce the number and size of gates that are needed to build a circuit.
- The minimization methods reduce the cost of a 2-level AND-OR, OR-AND, NAND-NAND, or NOR-NOR circuit by:
 - minimizing the number of first-level gates
 - minimizing the number of inputs on each first-level gate
 - minimizing the number of inputs on each second-level gate



Combinational Circuit Minimization

- **Most minimization is based on the following theorems:**
 $\text{product} \cdot Y + \text{product} \cdot Y' = \text{product}$ (**Adjacency or Uniting Theorem**)
 $(\text{sum} + Y) \cdot (\text{sum} + Y') = \text{sum}$
- If two terms differ only in the complementing (or not of) one variable, they can be combined into a single term with one less variable.



Combinational Logic Optimization and Tradeoffs

- **Two-level size optimization using algebraic methods**
 - Goal: circuit with only two levels (ORed AND gates), with minimum transistors
 - Though transistors getting cheaper (Moore's Law), they still cost something
- Define problem algebraically
 - Sum-of-products yields two levels
 - $F = abc + abc'$ is sum-of-products; $G = w(xy + z)$ is not.
 - Transform sum-of-products equation to have fewest literals and terms
 - Each literal and term translates to a gate input, each of which translates to about 2 transistors
 - Ignore inverters for simplicity

Example

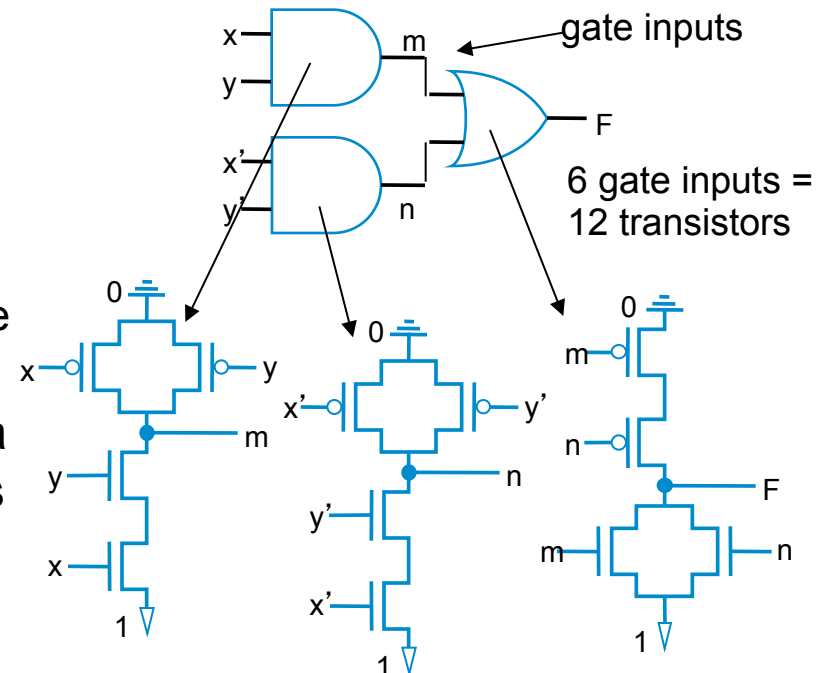
$$F = xyz + xyz' + x'y'z' + x'y'z$$

$$F = xy(z + z') + x'y'(z + z') \text{ (distr.)}$$

$$F = xy*1 + x'y'*1 \text{ (complement)}$$

$$F = xy + x'y' \text{ (ident.)}$$

4 literals + 2 terms = 6 gate inputs



Algebraic Two-Level Size Minimization

- Previous example showed common algebraic minimization method

- Multiply out to sum-of-products, then
- Apply Adjacency/Uniting Theorem (or axioms) as much possible

- $ab + ab' = a(b + b') = a \cdot 1 = a$
- “Combining terms to eliminate a variable”

- Duplicating a term sometimes helps
 - Note that doesn't change function
 - Idempotent

$$c + d = c + d + d = c + d + d + d + d \dots$$

- Sometimes after combining terms, can combine resulting terms

$$\begin{aligned} F &= xy\mathbf{z} + xy\mathbf{z}' + x'y'\mathbf{z}' + x'y'\mathbf{z} \\ F &= xy(\mathbf{z} + \mathbf{z}') + x'y'(\mathbf{z} + \mathbf{z}') \text{ (dist)} \\ F &= xy \cdot \mathbf{1} + x'y' \cdot \mathbf{1} \text{ (compl)} \\ F &= xy + x'y' \text{ (identity)} \end{aligned}$$

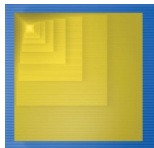
$$\begin{aligned} F &= x'y'z' + \mathbf{x'y'z} + x'yz \\ F &= x'y'z' + \mathbf{x'y'z} + \mathbf{x'y'z} + x'yz \text{ (idempotent)} \end{aligned}$$

$$F = x'y'(z+z') + x'z(y'+y) \text{ (distr)}$$

$$F = x'y' \cdot 1 + x'z \cdot 1 \text{ (complement)}$$

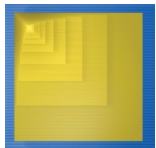
$$F = x'y' + x'z \text{ (identity)}$$

$$\begin{aligned} G &= xy'\mathbf{z}' + xy'\mathbf{z} + xyz + xy\mathbf{z}' \\ G &= xy'(\mathbf{z}' + \mathbf{z}) + xy(\mathbf{z} + \mathbf{z}') \\ G &= xy' + xy \text{ (now do again)} \\ G &= x(\mathbf{y}' + \mathbf{y}) \\ G &= x \end{aligned}$$



Covers

- An expression F **covers** another expression G , if the minterms in G are a subset of the minterms in F
 - Given three vars A, B, C ;
 - $A \cdot C$ covers $A \cdot B' \cdot C$, since $A \cdot C$ contains minterms 5 and 7
 - (since $A \cdot C = A \cdot C (B + B') = A \cdot C \cdot B + A \cdot C \cdot B' = A \cdot B \cdot C + A \cdot B' \cdot C$, from the set of 8 minterms on the variables A, B , and C)
 - and $A \cdot B' \cdot C$ contains only minterm 5.



General Simplification Procedure

Given an expression $F = (A \cdot B \cdot D + A' \cdot B + B \cdot C' \cdot D' + B' \cdot C \cdot D + B' \cdot C \cdot D')$

Step 1. Let M be the set of minterms covered by F

$$\begin{array}{cccc}
 & & A' \cdot B' \cdot C \cdot D & A' \cdot B' \cdot C \cdot D' \\
 A' \cdot B \cdot C' \cdot D' & A' \cdot B \cdot C' \cdot D & A' \cdot B \cdot C \cdot D & A' \cdot B \cdot C \cdot D' \\
 A \cdot B \cdot C' \cdot D' & A \cdot B \cdot C' \cdot D & A \cdot B \cdot C \cdot D & \\
 & & A \cdot B' \cdot C \cdot D & A \cdot B' \cdot C \cdot D'
 \end{array}$$

Step 2. For each minterm, m , find all *maximal* terms that cover m and also cover other minterms in M , but no minterms that are not in M .

Let T be the resulting set of terms.

$$T = \{A' \cdot B, B \cdot C', B \cdot D, C \cdot D, A' \cdot C, B' \cdot C\}$$

Step 3. Select all terms in T that cover minterms covered by no other terms in T . For example $ABC'D'$ minterm for BC' term. $\{B \cdot C', B' \cdot C\}$

Step 4. Select additional terms in T until selected terms cover all minterms. At each step, select a term that covers the largest possible number of new minterms. $\{A' \cdot B, C \cdot D\}$

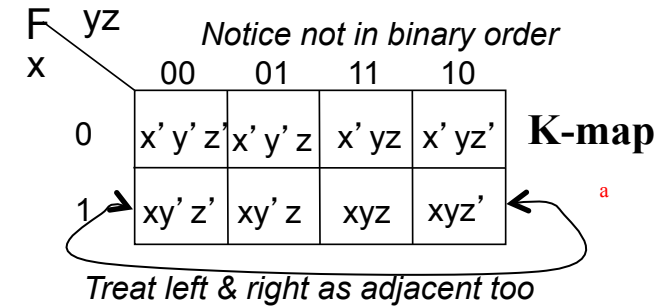
Finally: $\{B \cdot C', B' \cdot C, A' \cdot B, C \cdot D\}$



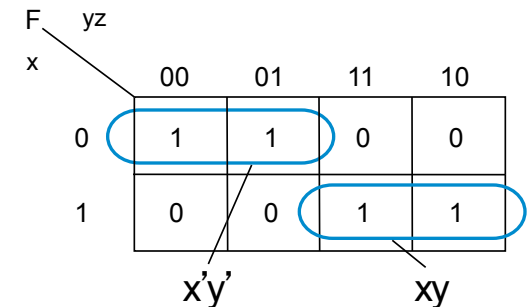
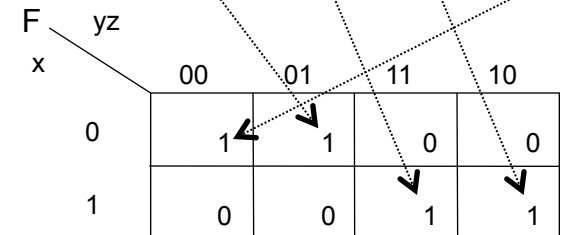
Karnaugh Maps for Two-Level Size Minimization

- **Karnaugh Maps (K-maps)**

- **Graphical** method to help us find opportunities to combine terms
- Minterms differing in one variable are *adjacent* in the map (can use adjacency theorem automatically)
- Can clearly see opportunities to combine terms – look for adjacent 1s
 - For F, clearly two opportunities
 - Top left circle is *shorthand* for $x' y' z' + x' y' z = x' y' (z' + z) = x' y' (1) = x' y'$
 - Draw circle, write term that has all the literals except the one that changes in the circle
 - Circle xy, x=1 & y=1 in both cells of the circle, but z changes (z=1 in one cell, 0 in the other)
 - Minimized function: OR the final terms
 - How to obtain POS representation from Kmap?

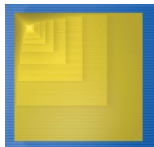


$$F = x' y' z' + x y z + x y z' + x' y' z'$$



$$\begin{aligned}
 F &= x y z + x y z' + x' y' z' + x' y' z \\
 F &= x y (z + z') + x' y' (z' + z) \\
 F &= x y \cdot 1 + x' y' \cdot 1 \\
 F &= x y + x' y'
 \end{aligned}$$

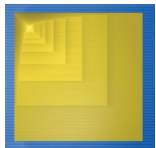
$$F = x' y' + x y$$



Example

- Use a K-Map to simplify the following Boolean expression

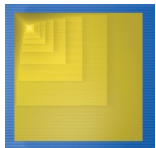
$$F(a,b,c,d) = \sum m(0,2,3,6,8,12,13,15)$$



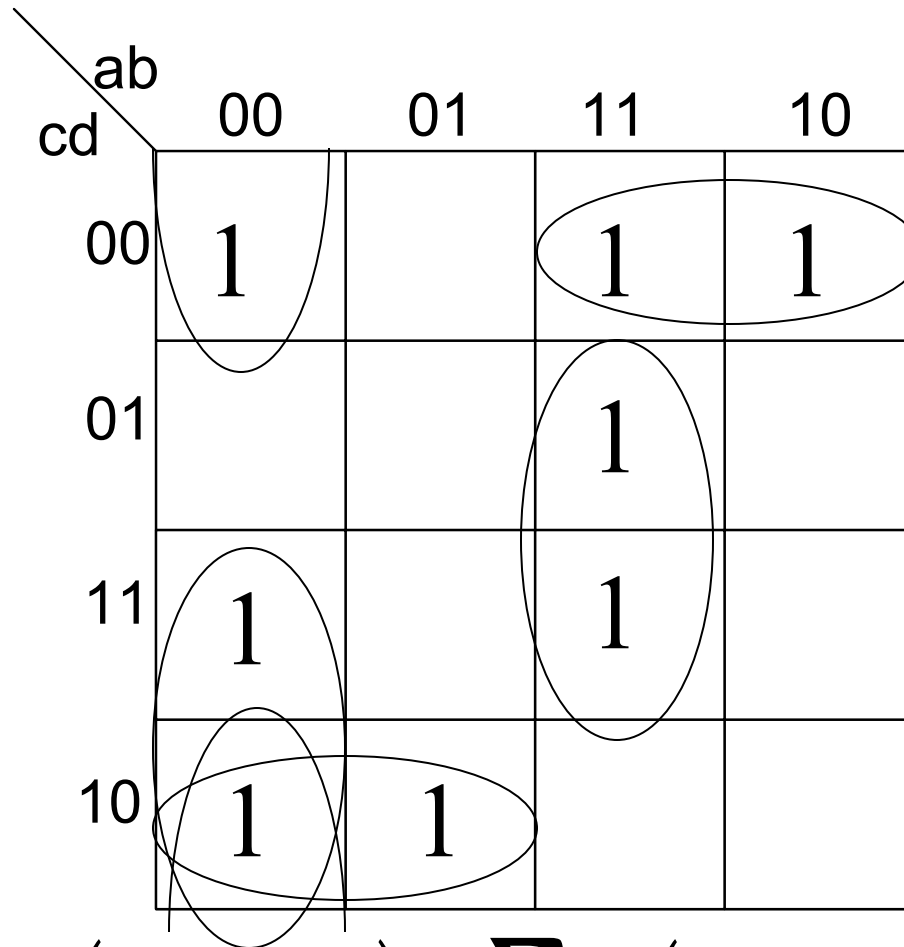
Four-variable K-Map

		ab			
		00	01	11	10
cd	00	1		1	1
	01			1	
	11	1		1	
	10	1	1		

$$F(a,b,c,d) = \sum m(0,2,3,6,8,12,13,15)$$



Four-variable K-Map

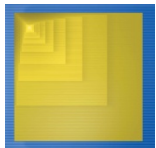


$$F(a,b,c,d) = \sum m(0,2,3,6,8,12,13,15)$$



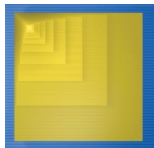
K-Maps

- K-maps are not commonly used in design practice.
- It is useful for two-level logic size optimization of three or four variable functions.
- We will mainly use Boolean Algebra and related theorems instead of K-maps.



Don't Care Input Combinations

- What if particular input combinations can never occur?
 - Minimize $F = xy'z'$, given that $x'y'z'$ ($xyz=000$) can *never* be true, and that $xy'z$ ($xyz=101$) can *never* be true
 - So it doesn't matter what F outputs when $x'y'z'$ or $xy'z$ is true, because those cases *will never occur*
 - Thus, make F be 1 or 0 for those cases *in a way that best minimizes the equation*
 - Unnecessary use of don't cares; results in extra term



Minimization Example using Don't Cares

- Minimize:

- $F = a'bc' + abc' + a'b'c$
- Given don't cares: $a'bc, abc$

$$F = a'bc' + abc' + a'b'c$$
$$= a'bc' + abc' + a'b'c + \mathbf{a'bc} + \mathbf{abc} + \mathbf{a'bc}$$

(adding don't cares)

$$= bc'(a' + a) + a'b'c + \mathbf{a'bc} + \mathbf{abc} + \mathbf{a'bc}$$

(distributive law)

$$= bc' + a'b'c + \mathbf{a'bc} + \mathbf{abc} + \mathbf{a'bc}$$

(complement,)

$$= bc' + a'b'c + \mathbf{a'bc} + \mathbf{bc(a+a')} \text{ (distributive)}$$

$$= bc' + a'b'c + \mathbf{a'bc} + \mathbf{bc} \text{ (complement)}$$

$$= b(c' + c) + a'b'c + a'bc \text{ (distr)}$$

$$= b + a'c(b' + b) \text{ (distr)}$$

$$= b + a'c \text{ (complement)}$$

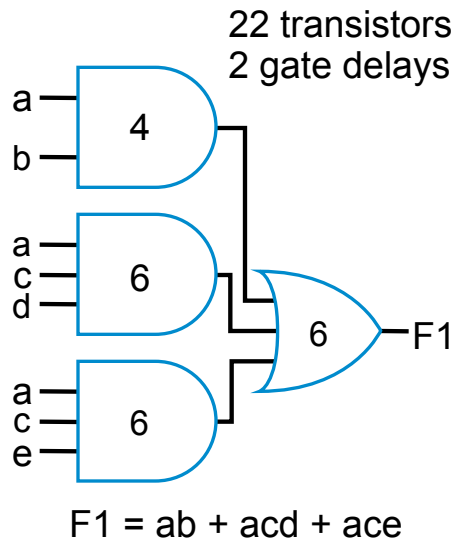
- Note: Use don't cares with caution

- Must be *sure* that we really don't care what the function outputs for that input combination
- If we do care, even the slightest, then it's probably safer to set the output to 0

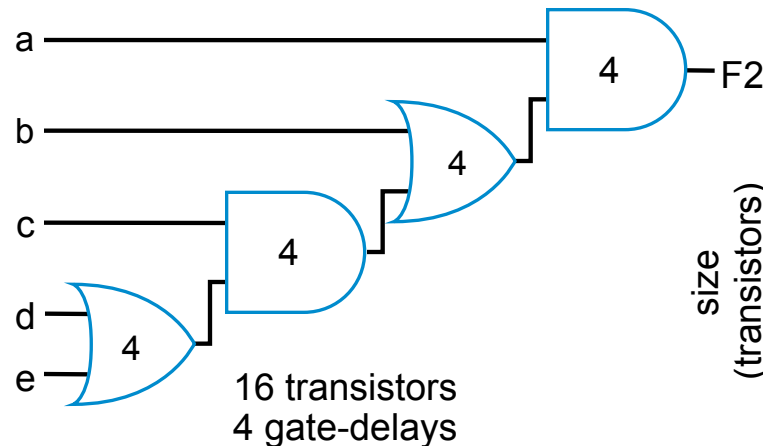


Multi-Level Logic Optimization – Performance/Size Tradeoffs

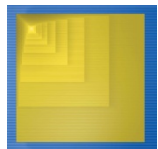
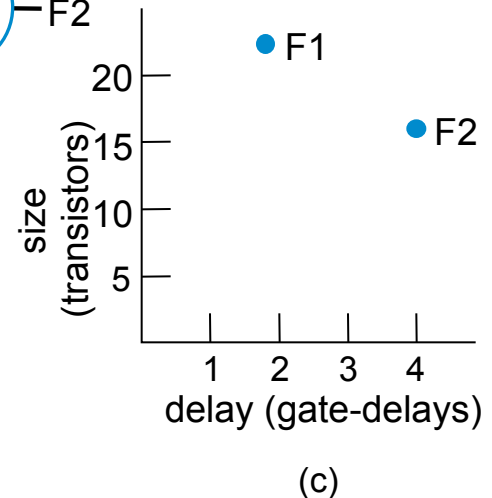
- We don't always need the speed of two level logic
 - Multiple levels may yield fewer gates
 - Example
 - $F1 = ab + acd + ace \rightarrow F2 = ab + ac(d + e) = a(b + c(d + e))$
 - General technique: Factor out literals – $xy + xz = x(y+z)$



(a)



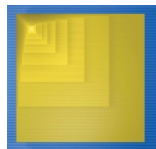
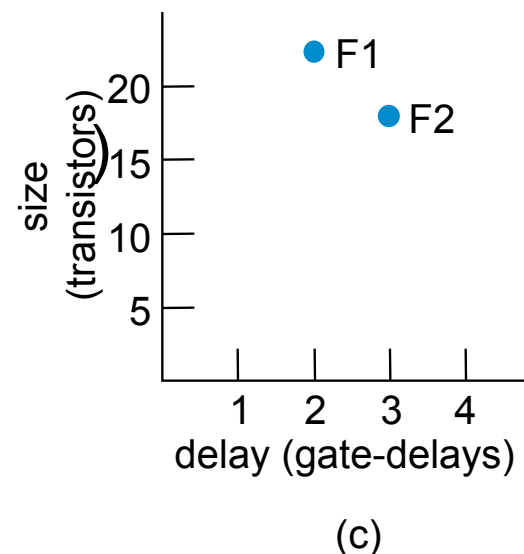
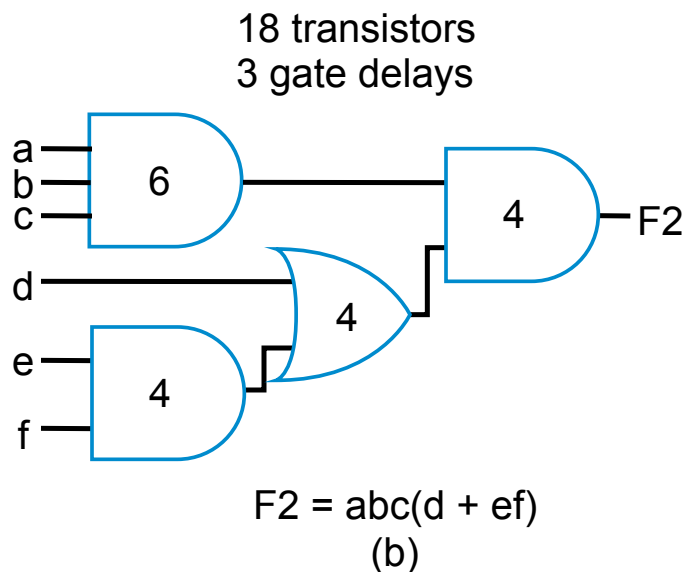
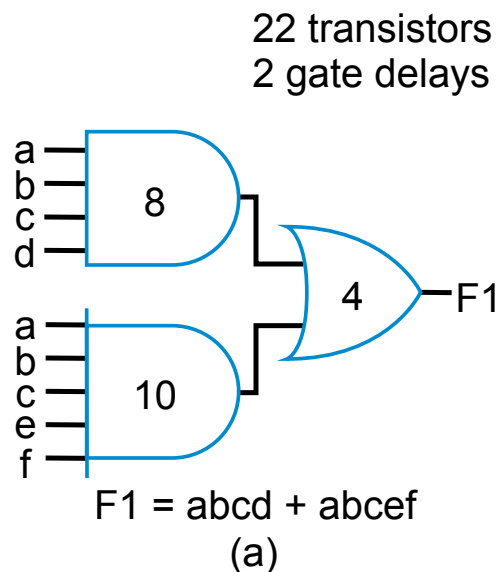
(b)



Multi-Level Example

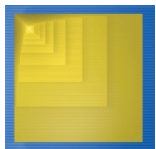
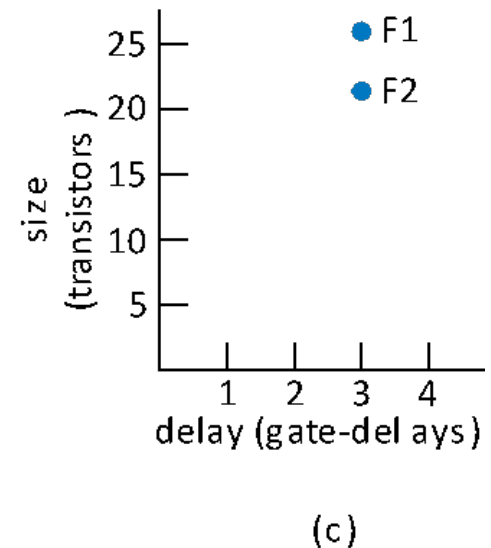
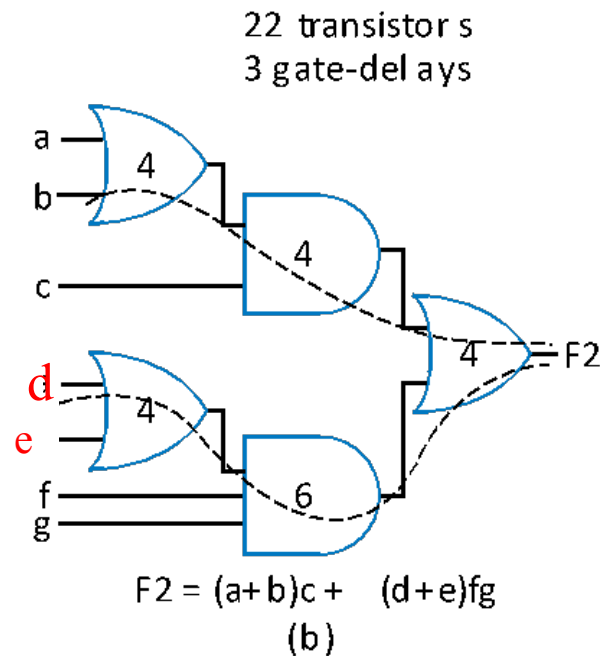
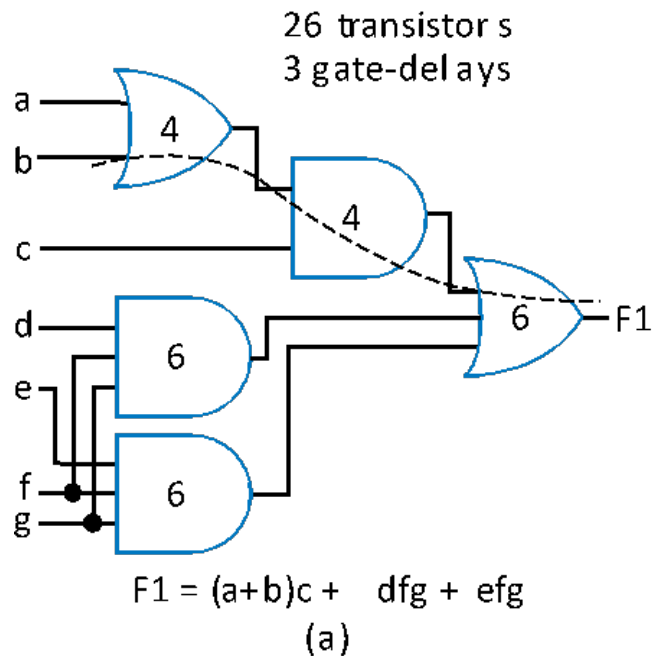
- Q: Use multiple levels to reduce number of transistors for
 - $F1 = abcd + abcef$
- A: $abcd + abcef = abc(d + ef)$
 - Has fewer gate inputs, thus fewer transistors

a



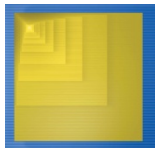
Multi-Level Example: Non-Critical Path

- **Critical path:** longest delay path to output
- Optimization: reduce size of logic on non-critical paths by using multiple levels



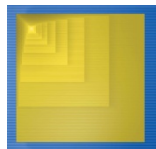
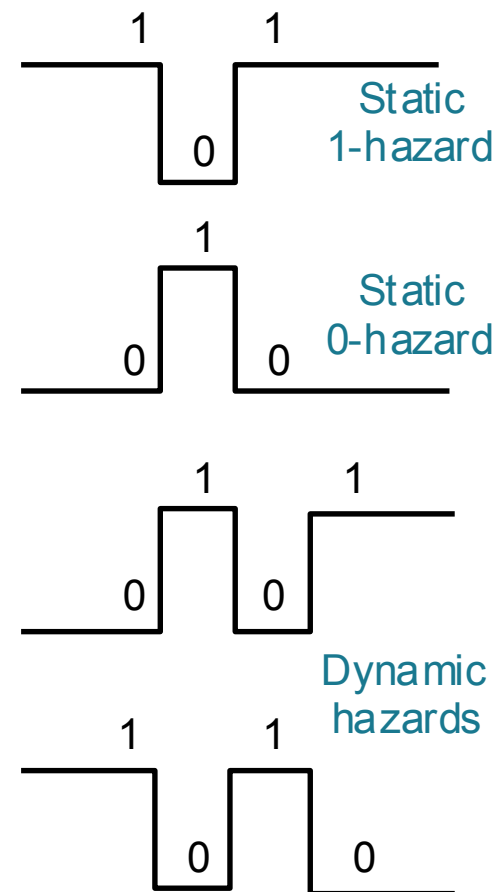
Automated Minimization Methods

- Main techniques use heuristic iterative methods (as opposed to exact methods where all minterms need to be explored leading to exponential complexity)
 - Define various operations
 - “Factor out”: $xy + xz = x(y+z)$
 - Expand, and others
 - Randomly apply, see if improves
 - May even accept changes that worsen, in hopes eventually leads to even better equation
 - Keep trying until can't find further improvement
 - Not guaranteed to find optimal circuit, but rather a good one



Hazards

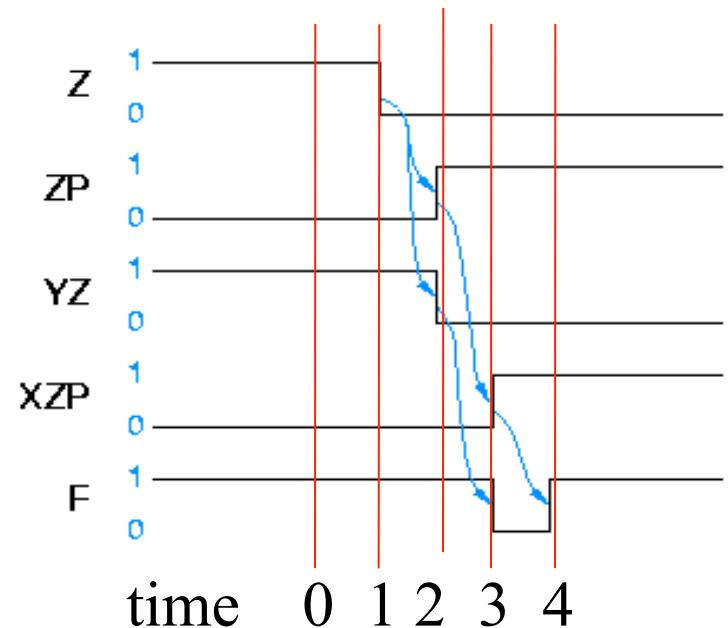
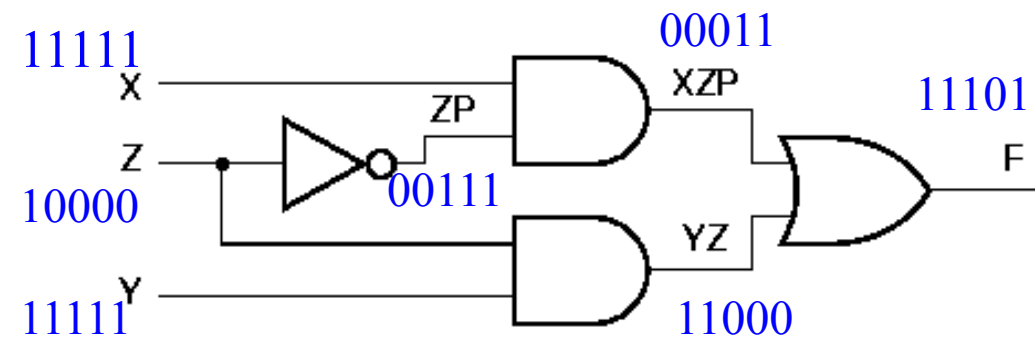
- Due to **delays** on electronic devices, a circuit may produce an undesirable short pulse, called a glitch.
- A hazard exists when a circuit has the possibility of producing a glitch.
- A static hazard occurs when it is possible for an output to undergo a momentary transition when it is expected to remain unchanged.
- A dynamic hazard occurs when the output has the potential to change more than once when it is expected to make a single transition.



Hazards (from Wakerly)

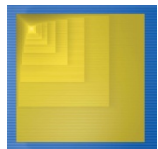
- A static-1 hazard is a pair of input combinations that differ only in one input variable, both give 1 as output, such that a momentary 0 output may occur, during a transition in the differing input variable.

Static-1 hazard occurs below, when $X=Y=1$ and Z is changing from 1 to 0.



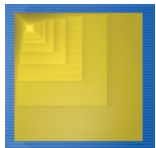
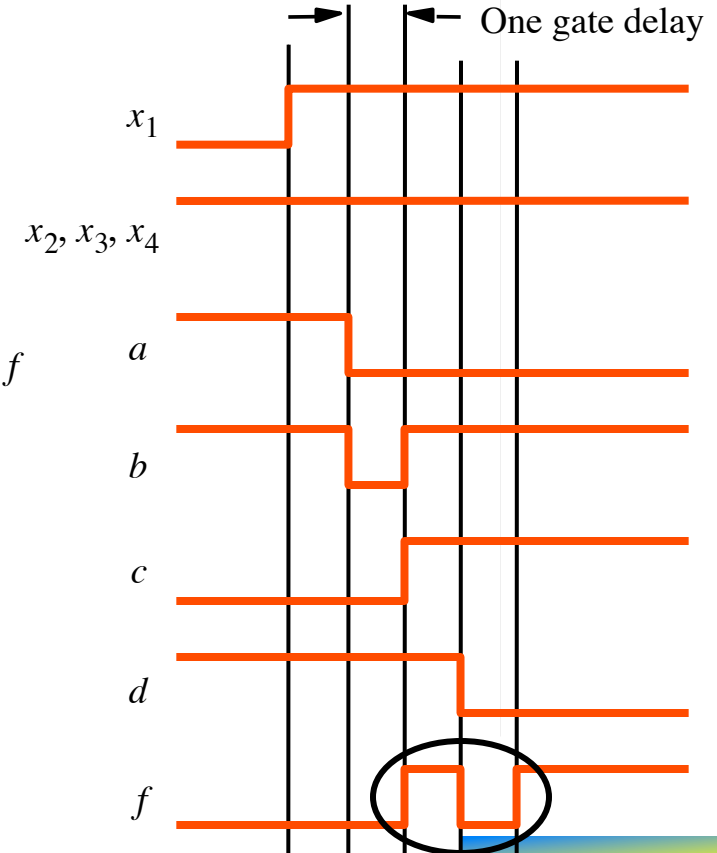
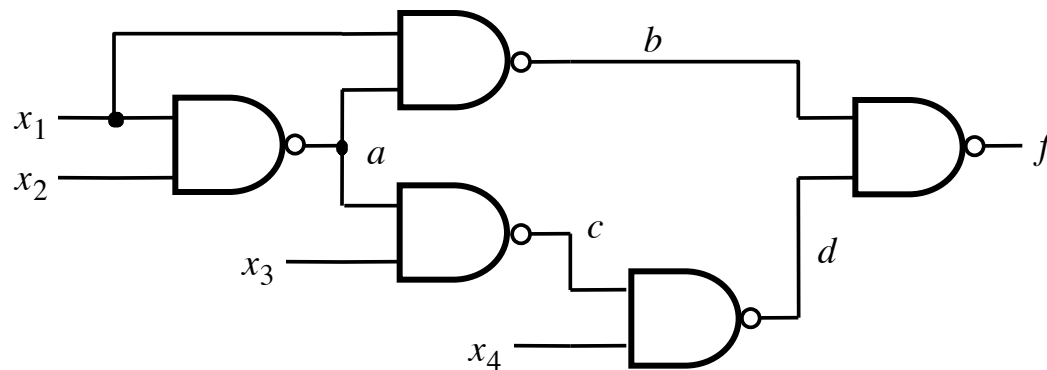
Assume unit delay.

Change in Z takes 3 time units to reach F from the top path, 2 from the lower path



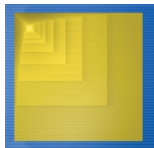
Dynamic Hazards

Theorem: Dynamic hazards do not occur in two-level combinational circuits → only in multi-level circuits



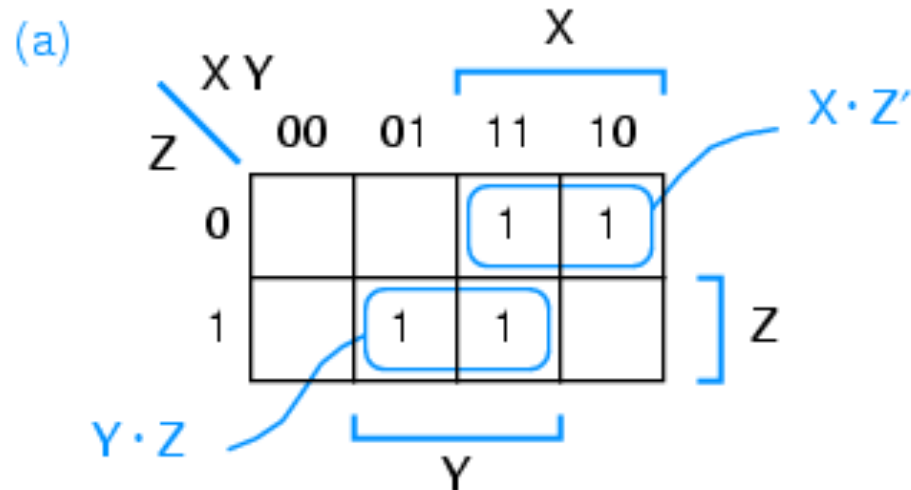
Detection of Static-1 Static-0 Hazards

- A properly designed AND-OR circuit will never have static-0 hazards but it may have static-1 hazards. Why?
- A static-0 hazard would exist in such a circuit only if a variable and its complement were connected to the same AND gate – which would not make sense.
- An OR-AND circuit may have static-0 hazards
- Hazards can be detected from a K-map. How?
- Check for adjacent minterms NOT covered by the same product term (AND gate) in the actual realization. **add redundant terms (for adjacent 1's that are not covered for static-1 hazards, dual for static-0)**



Detecting Hazards with K-maps

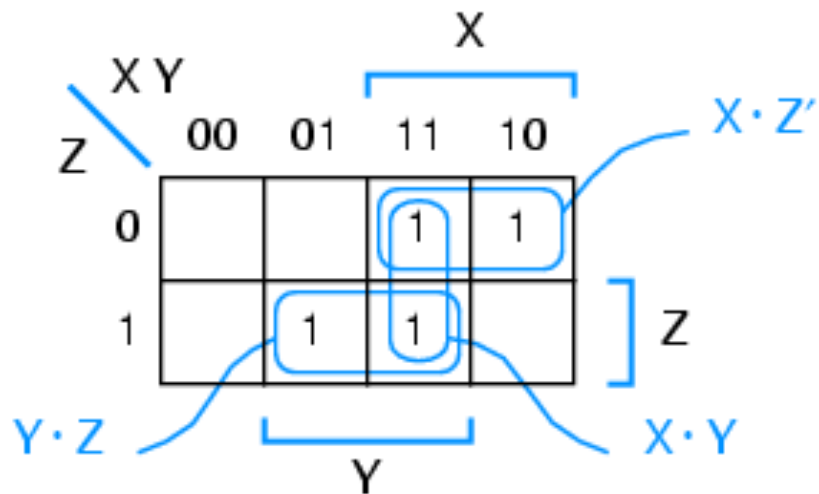
Correct any static-1 hazards in the following function
(circle on map and write new F function)



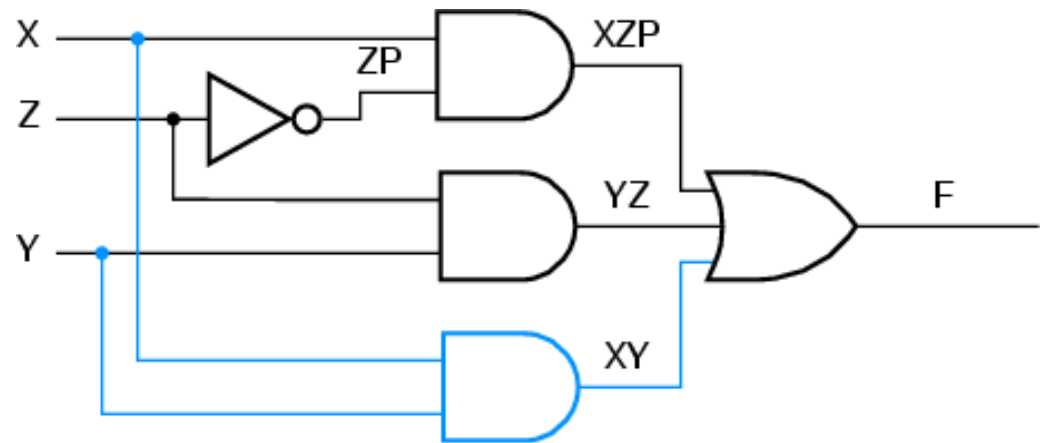
$$F = X \cdot Z' + Y \cdot Z$$



Detecting Hazards with K-maps



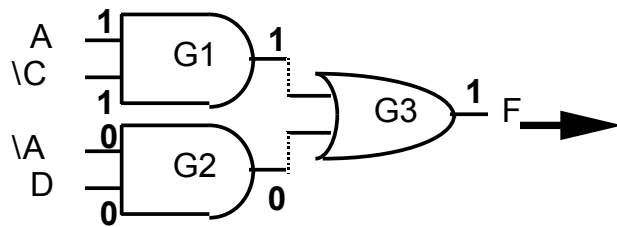
$$F = X \cdot Z' + Y \cdot Z + X \cdot Y$$



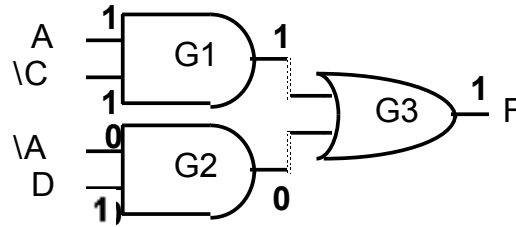
Hazards

- $F = \sum_{A,B,C,D} (1,3,5,7,8,9,12,13)$

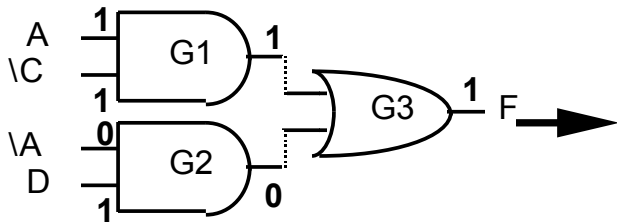
One input changed (D input), no glitch occurred



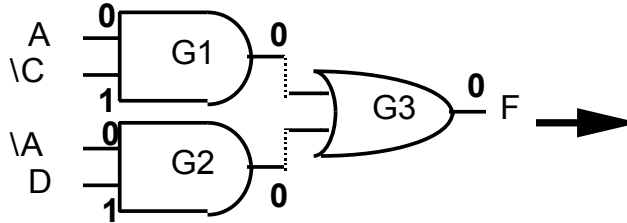
ABCD = 1100



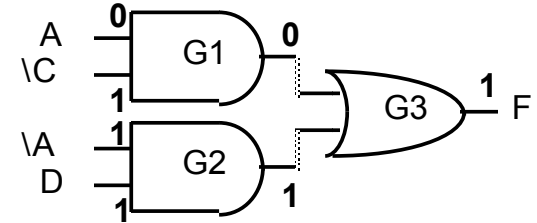
ABCD = 1101



ABCD = 1101



ABCD = 0101 ($\bar{A}$ is still 0)



ABCD = 0101 ($\bar{A}$ is 1)

One input changed (A input), glitch occurred

Exercise

- Detect the hazard in $F = (x + y) (y' + z)$ when $x=z = 0$ and y changes from 0 to 1. Obtain F from Kmap then correct the hazard using Kmap.

